Calyx: um compilador que exige o contrato de efeito em workflows de agentes

Dalton Fontes

https://github.com/daltonfontes/calyx

Uso de IA. A Calyx foi concebida e dirigida pelo autor, que leu os trabalhos relacionados e revisou cada mudança; o código, os experimentos e este texto foram escritos com o Claude (Anthropic) como assistente de programação. Tradução da versão em inglês (`paper/Calyx.pdf`).

Resumo

Workflows de agentes de LLM chamam modelos e tools cujos efeitos saem da execução: pagam, enviam e-mail, editam repositórios e esperam por pessoas. Frameworks de agentes como o LangGraph e motores de execução durável como o Temporal retomam uma execução interrompida, mas deixam o contrato de cada efeito para o programador: se ele pode ser repetido, com qual chave, e o que fazer quando ninguém sabe se ele aconteceu. Quando esse código falta, o resultado é um segundo pagamento, um e-mail repetido, uma atualização perdida.

A Calyx é uma linguagem de workflows em que toda tool declara seu efeito (`read`, `write` com chave de idempotência, ou `write once` com uma política para resultados incertos), e o compilador deriva o paralelismo e as verificações do grafo de dependências. Um programa que pode duplicar um efeito, perder uma atualização ou deixar duas escritas sem ordem é rejeitado ou recebe um aviso antes de rodar; o runtime grava cada chamada num diário e retoma sem refazer trabalho. Enunciamos as garantias como três teoremas sob cinco hipóteses explícitas, provamos em Lean os protocolos de uma chamada e verificamos programas inteiros exaustivamente num modelo limitado, que também produz um contraexemplo para cada hipótese retirada.

Contra o LangGraph e o Temporal nos mesmos workflows, a Calyx acerta todos os cenários de queda, espera e concorrência por padrão; os baselines só acertam com cuidado escrito à mão. No LIMBO, um benchmark externo de efeitos duplicados, programas Calyx empatam com os melhores modelos de fronteira sob os contratos nativos das tools (76% de sucesso com efeito único, contra 74–79%) e chegam a 100% quando toda escrita aceita chave. A garantia mora no contrato da tool; a Calyx torna o contrato obrigatório.

1 Introdução

Um agente de atendimento recebe um pedido de reembolso, propõe um valor, uma pessoa aprova, o agente paga e manda e-mail ao cliente. Se o processo morre depois de o pagamento ser feito mas antes de o resultado ser gravado, a execução retomada paga de novo? Se ele morre com o e-mail a caminho, o e-mail sai de novo? Se a aprovação chega depois do prazo, mas antes de alguém retomar a execução, ela vale?

Nos frameworks atuais essas perguntas têm resposta, mas a resposta é código que o programador escreve, ou esquece de escrever, sem nada que o lembre. Medimos isso (§6): com os padrões do LangGraph, a queda que a Calyx aguenta produz dois pagamentos; com o Temporal, que grava cada passo num histórico durável, um efeito a caminho no momento da queda é repetido.

A tese deste trabalho é que **o contrato de um efeito pertence à declaração da tool, e o compilador deve exigi-lo**. Os mecanismos de runtime já existem: diários por chamada (Temporal [21], Restate [17]), chaves de idempotência [9, 18] e conferir antes de reenviar [13]. Há também evidência independente de que o comportamento de efeito único depende do contrato da tool, não do modelo: o LIMBO [12] mostra que oferecer chave de idempotência em toda escrita corta muito as duplicatas, e que, para falhas que uma releitura não resolve (commits atrasados, reentrega), só o contrato ajuda. O que falta é algo que **exija** o contrato.

Nossas contribuições:

1. Uma linguagem para workflows de agentes (§3) cujo grafo de execução está implícito nas dependências de dados, e em que toda tool declara seu efeito e o que fazer com um resultado incerto.
2. Verificações do compilador (§4) que usam essas declarações para rejeitar programas que podem duplicar um efeito, perder uma atualização, deixar escritas sem ordem, esperar para sempre ou repetir um pagamento a cada volta de um laço; e um núcleo formal com três teoremas, provados em Lean para uma chamada e verificados exaustivamente num modelo limitado para programas inteiros.
3. Um runtime (§5) com um diário por chamada que retoma sem refazer trabalho e leva a chave de idempotência até o servidor da tool. Os mecanismos não são novos; o novo é um compilador que exige as declarações de que eles dependem.
4. Uma avaliação (§6) contra o LangGraph e o Temporal, com a matriz completa de quedas, prazos, memória compartilhada, custo do runtime, um estudo de bugs reais com resultado desfavorável e um benchmark externo (LIMBO).

2 O problema

Os bugs que atacamos não são de regra de negócio, mas de **estado**: o que acontece com os efeitos quando uma execução é paralela, interrompida, repetida ou concorrente. A Tabela 1 lista sete categorias, tiradas do estudo de issues da §6.6.

Categoria	Exemplo
Efeito repetido	Uma tarefa repetida paga de novo
Escrita concorrente	Dois ramos paralelos escrevem o mesmo campo
Atualização perdida	Duas execuções leem um saldo, somam e gravam
Espera	Uma aprovação sem prazo; uma resposta aplicada duas vezes
Laço de agente	O agente repete a mesma chamada até o limite
Ordem de efeitos	O e-mail de confirmação sai antes do pagamento
Recuperação	Retomar refaz trabalho ou restaura estado inconsistente

Tabela 1: Bugs de estado em workflows de agentes.

O que esses bugs têm em comum é que o programa não diz o que precisa ser dito: que o pagamento não pode ser repetido sem chave, que o e-mail precisa de uma regra para quando ninguém sabe se ele saiu, que o e-mail vem depois do pagamento. Em Python, essa informação não tem onde morar para que uma ferramenta a confira.

3 A linguagem

Um programa Calyx declara modelos, tools, prompts tipados e grafos. A sintaxe lembra Python; a semântica é de fluxo de dados.

```
tool refund(request: Text, order: Text,
            amount: Float) -> Unit:
    effect write
    idempotency_key request
    checks OrderState

tool email(to: Text, subject: Text,
           body: Text) -> Unit:
    effect write once
    timeout 10 s
    on_uncertain verify(email_sent(to, subject))

graph handle_refund(request: Text, order: Text,
                    message: Text) -> Text:
    found = get_order(order)
    proposal = gemini(decide(found, message))

    paid = refund(request, order, proposal.amount):
        requires state.status == Delivered
        requires state.refunded + proposal.amount
                 <= state.total

    body = gemini(reply(found, proposal.amount))
    notice = email(found.email, "Refund {order}", body)
    notice after paid

    return "refunded {proposal.amount}"
```

Listagem 1: O reembolso em Calyx. Nenhuma linha diz o que roda em paralelo nem o que fazer depois de uma queda.

Grafo implícito. Cada `nome = valor` é um passo, e um passo depende dos passos que usa. O compilador ordena os passos pelas dependências e roda os independentes em paralelo (na Listagem 1, `reply` pode rodar junto com `refund`). `after` acrescenta uma ordem sem dados.

Efeitos declarados. Uma tool é `read`, `write` ou `write once`. Um `write` pode ser repetido se tiver chave de idempotência. Um `write once` nunca é repetido sozinho e precisa dizer o que fazer quando seu resultado é incerto: `pause` (uma pessoa decide ao retomar), `accept_loss`, ou `verify(f(...))`, em que `f` é uma tool `read` que encontra a chamada. `f` pode devolver `Bool`, ou uma lista do que a chamada criou, reencontrado; então o primeiro item vira a resposta da chamada (o id do ticket criado, por exemplo). Um `write once` que aplica uma lista de itens um a um declara `batch itens`; seu `verify` devolve os itens já aplicados, e o runtime reenvia só o resto. `requires` envia precondições ao servidor da tool, que as confere no mesmo passo do efeito.

Políticas. Uma `policy` de uma tool declara regras que o runtime confere antes de toda chamada, de um grafo ou de um agente: `require amount > 0`; `require order from get_ticket.order`, o argumento precisa ser um valor que uma tool devolveu nesse campo, não um que só aparece num texto que o modelo leu; `deny in agent if amount > 100`. Uma chamada recusada não é feita; num agente, vira uma observação para o modelo. Uma tool que a política nega sempre a agentes não pode ser dada a um (`E0724`).

Outras construções, cada uma com suas regras no compilador: `loop` e `rounds` com limite obrigatório; `for each` (leque de chamadas); `race` entre estratégias, cancelando as perdedoras e desfazendo o que elas escreveram (uma tool pode declarar `compensate g(...)`, uma escrita com chave que a desfaz: uma saga, guardada no diário); `agent` (o ciclo ReAct [24]), que só recebe tools sem efeitos irreversíveis; `entity`, estado compartilhado entre execuções com handlers puros; `receive`, uma espera durável por uma mensagem de fora com prazo obrigatório; e um roteador de modelos que tenta primeiro o mais barato.

4 O que o compilador verifica

As verificações usam três coisas: o tipo de cada valor, o efeito de cada passo e o grafo de dependências. A Tabela 2 lista as que tratam de estado.

Código	Rejeita ou avisa sobre
`E0304`	`write once` sem política para resultado incerto
`W0601`	`write` sem chave de idempotência
`W0602`	Duas escritas externas sem ordem definida
`W0603`	Um `send` a uma entidade calculado a partir de um `ask` a ela, quando o handler grava um valor novo (atualização perdida)
`W0604`	Um ramo de `race` que escreve sem `compensate`
`E0695/6`	Uma compensação que não é escrita com chave, ou usa outros valores que não os da chamada
`W0605`	Um `write once` num laço cujos argumentos não mudam entre as voltas
`E0640`	Um agente que recebe uma tool `write once`
`E0671`	`receive` sem prazo
`E0637`	Um `batch` que não é lista, ou não é verificado
`W0701–3`	Uma declaração que o servidor MCP da tool contradiz (`check --tools`)

Tabela 2: Verificações de estado (parcial; a especificação lista todas).

Várias vieram da avaliação: `W0605` do estudo de bugs reais (§6.6), `E0637` e a forma de lista do `verify` do LIMBO (§6.7), e `W0701–3` da observação de que uma declaração errada é o ponto mais fraco (§6.8).

4.1 O que as regras garantem

Formalizamos um núcleo da linguagem: chamadas a modelos, `read`, `write key p`, `write` sem chave e `write once` π; um diário dividido entre o que está no disco e o que só o sistema operacional guarda; falhas de transporte (pedido perdido, resposta perdida); e quedas do processo ou da máquina. Retomar é rodar o programa de novo com o mesmo diário. As regras são:

- **R0.** Uma chamada cuja entrada no diário é `done(κ, v)` devolve `v` sem ser feita.
- **R1.** Uma leitura ou chamada de modelo é feita, repetindo erros temporários, e gravada sem sincronizar.
- **R2.** Uma escrita com chave sincroniza o diário, envia com a chave `k = ⟦p⟧`, repetindo com a mesma chave, e grava `done` sincronizado.
- **R3.** Um `write once` grava `begin(κ)` sincronizado, envia e, num erro incerto, aplica sua política **D1**: aceitar, verificar (feito se encontrou, reenvia se não) ou parar para uma pessoa.
- **R4.** Ao retomar, `begin(κ)` sem `done` é incerto e vai para D1.

Os teoremas valem sob cinco hipóteses: (H1) o diário respeita o `fsync`; (H2) um serviço declarado com chave a respeita; (H3) o `verify` é atual: vê toda escrita confirmada, e nenhum pedido a caminho é confirmado depois de ele ler; (H4) o transporte entrega cada pedido no máximo uma vez; (H5) uma pessoa que retoma uma pausa decide de acordo com o que aconteceu.

T1 (no máximo uma vez). Sob H1–H5, para qualquer sequência de falhas de transporte, quedas do processo ou da máquina e retomadas, cada `write once` e cada escrita com chave é aplicado no máximo uma vez por chave do diário.

T2 (exatamente uma vez ao terminar). Se a execução termina, cada escrita com chave e cada `write once` com `verify` ou `pause` foi aplicado exatamente uma vez (no máximo uma vez com `accept_loss`).

T3 (nada feito é refeito). Uma chamada com `done` no disco nunca é enviada de novo, e retomar usa a resposta gravada. Só H1 é necessária.

Para uma chamada, as provas são mecanizadas em Lean 4 (`formal/Effects.lean`, conferido na CI): o `write once` e a escrita com chave são sistemas de transições em que cada escolha do mundo é um passo, e um invariante indutivo dá T1–T3 para toda execução; dois contraexemplos mostram a duplicata quando o runtime não sincroniza. Para programas inteiros o argumento é um esboço: T1 decorre de o registro `begin` chegar ao disco antes de qualquer envio, de modo que todo reenvio passa por D1, que só reenvia quando nada foi aplicado (H3, H5); para escritas com chave, a sincronização de R2 torna a chave uma função do estado durável, então ela é a mesma em toda tentativa, e H2 faz o resto. Também verificamos os teoremas exaustivamente num modelo executável limitado das regras (Tabela 3): um programa de sete chamadas com um modelo, escritas com chave (uma cuja chave vem da resposta do modelo), `write once` com cada política e uma leitura, sob toda combinação de até duas falhas de transporte e duas quedas, com toda retomada.

Caso	Execuções	Violações
Hipóteses valem, quedas do processo	53.166	**0**
Hipóteses valem, quedas da máquina também	217.928	**0**
Sem sincronizar antes de escrita com chave (bug corrigido)	257.488	18.500
`begin` fora do disco, quedas da máquina	268.360	49.346
Serviço ignora chaves (sem H2)	53.166	24.524
Releitura velha, commit atrasado (sem H3)	101.662	11.248
Transporte reentrega (sem H4)	95.556	24.856

Tabela 3: Verificação exaustiva limitada de T1–T3 (`bench/formal/model.py`).

As três últimas linhas são, no modelo, exatamente as três causas de duplicatas que o LIMBO mediu na Calyx (§6.7). Escrever a regra R3 com H1 explícita também expôs dois buracos no runtime, já corrigidos: um `write once` saía mesmo se seu registro `begin` não chegasse ao disco, e uma escrita com chave não sincronizava o diário antes, então, depois de uma queda da máquina, um modelo perguntado de novo podia produzir outra chave (a terceira linha).

5 O runtime

O compilador emite uma representação intermediária em JSON que um runtime escrito em C executa, com uma camada de E/S em Rust (APIs de modelos, clientes MCP, sandboxes).

Diário por chamada. Toda chamada tem uma chave estável, seu lugar no grafo realizado (passo, volta do laço, item do `for each`), e toda resposta vai para um diário só de acréscimo. Antes de qualquer escrita externa, o diário é sincronizado com o disco; um `write once` também grava `begin` antes, de modo que achar `begin` sem resposta significa que o resultado é incerto, e a política da tool decide.

A chave chega ao servidor. A chave de idempotência e as precondições (`requires`) viajam até o servidor da tool nos metadados da chamada MCP, então nem uma repetição causada por um bug do runtime paga duas vezes. Um servidor na frente de outro serviço informa os erros temporários desse serviço com um texto de erro que começa com `Timeout:` ou `Unavailable:`, e um `write once` então aplica sua política em vez de falhar.

Entidades. O estado de uma entidade fica num arquivo, mudado por uma mensagem por vez por chave (`flock`), com o id de cada mensagem aplicada guardado junto: uma mensagem nunca é aplicada duas vezes, mesmo que a execução morra entre a entidade aplicá-la e o diário registrá-la.

Esperas. Um `receive` grava seu prazo absoluto uma vez; a execução sai (estado `waiting`) e um agendador (`calyx tick`, rodado pelo cron) a retoma quando a mensagem chega ou o prazo passa. A hora de chegada é gravada, e uma resposta atrasada não vale mesmo que ninguém tenha retomado a execução ainda (§6.3).

Declarações contra servidores. Servidores MCP podem anotar tools com dicas (`readOnlyHint`, `idempotentHint`) [14]. Na primeira chamada de cada tool, e em `calyx check --tools` sem rodar, a Calyx as compara com a declaração e avisa numa contradição (§6.8).

Várias máquinas. Com o diário no PostgreSQL, as mesmas linhas vão para uma tabela, cada uma confirmada antes da próxima chamada; uma execução roda sob um **advisory lock** de sessão com o id dela, que o PostgreSQL solta com a sessão de um processo que morre, então um worker em outra máquina assume a execução e a termina a partir do diário. Uma entidade é uma linha: uma mensagem a trava, roda o handler e grava o novo estado com os ids das mensagens aplicadas numa transação, então o exatamente-uma-vez vale entre máquinas; as esperas e mensagens de `receive` também são linhas.

Programas como servidores. `calyx serve` transforma um programa num servidor MCP: cada grafo é uma tool, anotada a partir do programa (`readOnlyHint` quando nenhum passo escreve, senão `idempotencyKeyHint`). A dica da chave vale por construção: uma chamada com chave fica presa a uma execução, e a mesma chave devolve a resposta dela, ou a retoma se ela morreu, em vez de rodar de novo.

6 Avaliação

Perguntamos: o paralelismo derivado é bom (Q1)? O compilador pega bugs de estado que o Python não pega (Q2)? A recuperação é correta sem código escrito à mão (Q3)? O custo do runtime é desprezível? Os baselines são Python sequencial, asyncio escrito à mão, LangGraph 1.2.12 [11] (`durability` padrão e `sync`) e Temporal (SDK Python 1.34.0, servidor 1.32.0) [21]. Todos rodam contra o mesmo mundo falso: modelos com latência fixa e as mesmas tools MCP (uma loja com pagamentos e e-mail). "Cuidado manual" é o código que a documentação de cada sistema recomenda e que um programador atento escreve: chaves de idempotência, conferir antes de reenviar, um prazo guardado no estado. Dois experimentos foram repetidos com um modelo real (Gemini). Código, dados e os comandos para refazer tudo estão no repositório (`bench/`).

6.1 Paralelismo e custo (W1, E2)

Um leque de N perguntas (busca + resumo cada) e um relatório, no máximo 8 chamadas ao mesmo tempo. Com cada chamada de modelo levando 1 s, a Calyx fica a 30–50 ms do limite teórico

(⌈N/8⌉+1 s), o asyncio escrito à mão a 80–95 ms e o LangGraph cerca de 0,8 s atrás (quase tudo na partida). Com o modelo real (Tabela 4) o quadro se mantém: a Calyx empata com o asyncio. Sem latência, o custo da Calyx é linear até 10^5 itens, 0,12–0,16 ms por item, com o diário custando até 14%; o asyncio é cerca de 5× mais barato por item (a Calyx faz uma chamada MCP de verdade por item) e o LangGraph chega a 7 ms por item e cresce.

Sistema	N = 5	N = 10
Calyx	**5,1 s**	**6,1 s**
Python asyncio (à mão)	5,2 s	6,4 s
LangGraph	7,3 s	8,1 s
Python, sequencial	10,6 s	17,3 s

Tabela 4: W1 com `gemini-3.5-flash-lite`, mediana de 3 execuções. N é pequeno por causa do limite de requisições da chave de teste.

Medir o custo também achou um bug na Calyx: o passo de um agente era reavaliado desde a primeira volta a cada resposta, um custo cúbico no número de voltas. Guardar o progresso do agente entre avaliações o corrigiu (400 voltas: de 70 s para 1,5 s); o que resta é quadrático e vem do protocolo de chat, que envia a conversa inteira a cada volta.

6.2 Recuperação com efeitos externos (W2)

O reembolso da Listagem 1, com o processo morto em seis pontos: depois de cada passo gravado, e com cada efeito a caminho (a loja pagou ou enviou, a resposta não chegou). Certo é um pagamento, um e-mail e nenhuma chamada de modelo refeita.

Sistema	Certos	Com cuidado
Calyx	**6/6**	—
Temporal	4/6	6/6
LangGraph `durability="sync"`	4/6	6/6
LangGraph padrão	3/6 (2 chamadas de modelo refeitas)	6/6
Python, sem checkpoint	2/6	6/6

Tabela 5: W2, recuperação depois de `kill -9`. Idêntico, caso a caso, com o modelo real.

Entre passos, o Temporal e o LangGraph `sync` acertam, como a Calyx. Com um efeito a caminho, nenhum registro por passo ajuda, nem o histórico do Temporal: o efeito aconteceu e a resposta se perdeu, e a activity é repetida depois do seu timeout, como deve ser. Só o contrato do efeito evita a duplicata (uma chave que o provedor respeite, ou conferir antes de reenviar), e nos baselines ele é opcional. Com o modelo real as contagens são as mesmas; a Calyx retoma em no máximo 2,7 s, o Temporal em 11–16 s com os timeouts padrão.

Contra um serviço real. Repetimos a matriz de quedas contra a API do Stripe em modo de teste (`bench/stripe/`): um reembolso no Stripe, cuja chave o servidor de tools da Calyx repassa ao `Idempotency-Key` do próprio Stripe, e um crédito na conta do cliente (uma transação de saldo), que o Stripe não deduplica e a Calyx declara `write once` com `verify`. Morto depois de cada passo e com cada efeito a caminho, a Calyx fez exatamente um reembolso e um crédito nos cinco casos, contados pelo próprio Stripe, em duas rodadas. O mesmo programa sem os contratos fez 3/5: um reembolso e um crédito duplicados, ambos com o efeito a caminho.

6.3 Esperando uma pessoa, com prazo (W3)

O reembolso com aprovação humana e prazo, com o processo parado durante a espera, em seis cenários.

Cenário	Calyx	LangGraph	Temporal
Aprovado	✓	✓	✓
Prazo passa, tudo parado	✓	espera para sempre	✓
Resposta enviada duas vezes	✓	✓	✓
Resposta atrasada, depois da recusa	✓	paga	✓
Resposta atrasada, antes de retomar	✓	paga	paga
Resposta no prazo, retomada depois	✓	✓	✓

Tabela 6: W3. Com cuidado manual, LangGraph e Temporal fazem 6/6.

O LangGraph não tem prazo para um `interrupt()`. O Temporal tem, mas, sem worker rodando no prazo, o timer e o sinal atrasado chegam juntos ao próximo worker e o SDK entrega o sinal primeiro. **A Calyx tinha o mesmo bug**, achado por esta medida e corrigido (a hora de entrega é gravada e comparada com o prazo); antes da correção ela fazia 5/6.

6.4 Memória compartilhada (W7)

Uma volta de conversa que acrescenta fatos à memória de um usuário, com execuções simultâneas e quedas entre guardar a memória e registrá-la. A Calyx faz 3/3. O `Store` do LangGraph perde atualizações com 20 execuções simultâneas (em 3 de 6 rodadas, perdendo 1 a 3 conversas), grava de novo depois de uma queda e retomada, e repete 30 fatos quando 10 execuções caem e retomam; com cuidado manual (um item por fato, com o id da mensagem como chave) faz 3/3.

6.5 Várias máquinas (W8)

Com o diário no PostgreSQL, as execuções passam de uma máquina para outra (§5). A W8 mede quanto isso custa, contra o PostgreSQL 16 na mesma máquina e a alguns milissegundos (um proxy que atrasa cada byte; 2,8 e 7,0 ms de ida e volta), mediana de 5 (`bench/run_w8.py`).

Diário	Local	2,8 ms	7,0 ms
Arquivo	0,12 s	—	—
PostgreSQL, um commit por linha (v0.3.5)	0,85 s	6,9 s	15,8 s
PostgreSQL, **group commit**	**0,15 s**	**0,24 s**	**0,37 s**

Tabela 7: W8, o fan-out da W1 com 500 perguntas e modelos que respondem na hora (1.003 linhas de diário de 8 threads).

A primeira versão confirmava cada linha numa transação, sob o lock do diário: uma ida e volta por linha. Agora uma thread confirma numa transação só todas as linhas escritas enquanto isso, e o runtime espera por ela antes de toda escrita externa e no fim, a regra do diário em arquivo (`fsync` no máximo uma vez por segundo, sempre antes de uma escrita). Um agente de 100 voltas (uma thread) cai de 3,5 s para 0,43 s a 7,0 ms; o arquivo leva 0,25 s.

Assumir uma execução. A máquina A morre no meio do reembolso da W2; um `calyx worker` em B a termina, com um pagamento e um e-mail. Quando o processo de A é morto, o sistema operacional fecha a conexão e B tem a execução 1,3 s depois (quase tudo é o intervalo de consulta de B). Quando a máquina de A some sem avisar (derrubamos todo pacote da conexão, como uma queda de energia faria), o servidor só descobre pelo **keepalive** do TCP, duas horas por padrão: a v0.3.5 não assumiu a execução em 60 s. Agora cada conexão ajusta o **keepalive** da própria sessão (5 s, depois 3 sondas a cada 2 s): 11,5 s.

Entidades. Oito processos em duas máquinas mandam 50 mensagens cada a uma entidade: nenhuma se perde (400 de 400). Uma mensagem são duas idas e voltas, com a linha travada durante

uma: 116 mensagens/s a 7,0 ms, 620 a 750 com o banco na máquina, 740 a 880 em arquivos numa máquina só. Listar 531 execuções (`calyx runs`, o que o worker faz a cada passada) leva 0,13 s; a primeira versão lia todos os diários, 0,90 s.

6.6 Bugs antes de rodar (Q2)

Um corpus de 54 bugs de estado, cada um um programa Calyx com o resultado esperado no cabeçalho: o compilador pega 35, 2 nem podem ser escritos, o runtime pega 11 e 6 escapam; 48 dos 54 nunca causam dano. Dos 16 portados para Python tipado + LangGraph, o pyright e o mypy pegam 2 dos 14 que a Calyx rejeita, e o LangGraph para 3 ao rodar, 2 deles depois do dano. O corpus e as portas foram escritos pelo autor do compilador, que é a principal ameaça a este resultado. Um kit para uma porta independente (E4) está em `bench/e4_porting/`; ele ainda não foi usado.

6.7 Bugs reais

Para não depender de bugs escritos pelo autor, buscamos nas issues do LangGraph, do CrewAI e do AutoGen [23] com oito buscas fixadas antes de ler os resultados, e classificamos cada relato pelo que a Calyx faria com o mesmo workflow, ficando com a classe menos favorável à Calyx na dúvida. Das 79 issues, 44 se qualificaram.

Classe	Issues
Bug do próprio framework (neutro)	29
A Calyx evita ao rodar	5
Não pode ser escrito em Calyx	2
A Calyx deixa passar	**8**
O compilador pega	**0**

Tabela 8: 44 bugs reais de issues do LangGraph, CrewAI e AutoGen.

O compilador não pegou nenhum. Issues relatam o framework errando, não o engano do programador: quem esquece a chave de idempotência não abre issue, encontra o pagamento duplo em produção. Issues públicas não medem bem a Q2, e não a confirmam. O estudo achou uma lacuna real (um `write once` dentro de um laço de nova tentativa paga a cada volta), que virou o aviso `W0605`; as contagens acima são da Calyx antes do estudo.

6.8 Um benchmark externo: LIMBO

O LIMBO [12] injeta falhas em seis serviços simulados e confere, num livro-razão, o que cada um de fato fez. Escrevemos suas 12 tarefas como programas Calyx, declarando as tools só a partir da documentação que um agente vê, e as rodamos na sua grade E2 (205 episódios em que a falha disparou) com o injetor de falhas e o avaliador do próprio LIMBO, sem mudanças. Não há modelo: cada tarefa é um programa fixo, então o que se compara é a recuperação. Os números dos modelos são os que o LIMBO publica.

	EOS	Dup.	TS
Calyx, contrato nativo	**76%**	**24%**	**100%**
3 modelos de fronteira, sem ajuda	74–79%	20–26%	99,5–100%
Melhor harness ciente do contrato	77%	23%	100%
Oráculo do resultado	88%	12%	100%
Calyx, chave em toda escrita	**100%**	**0%**	**100%**

Tabela 9: Grade E2 do LIMBO. EOS: sucesso com efeito único; Dup.: episódios com duplicata; TS: sucesso da tarefa.

Sob o contrato nativo (só os pagamentos e uma rede social aceitam chave), a Calyx empata com os melhores modelos e não os supera. Todas as suas duplicatas vêm de dois modos de falha que nenhum cliente corrige sem chave: a reentrega pelo transporte (74% para todo sistema, o oráculo incluído) e os commits atrasados, em que a escrita ainda está a caminho quando a releitura olha (71%; o melhor harness tem 69%). Onde uma releitura resolve a falha, a Calyx nunca duplicou. Quando toda escrita aceita chave, as duplicatas somem por construção: a chave está na declaração da tool, não na escolha do modelo a cada chamada. Isso apoia, de fora, a conclusão do LIMBO de que a garantia mora no contrato da tool; a contribuição da Calyx é tornar esse contrato escrito, verificado e igual em toda execução, sem custo de tokens.

O LIMBO achou quatro lacunas na Calyx, todas corrigidas antes destes números: o `verify` só dizia se uma escrita aconteceu, sem devolver o que ela criou (agora devolve, o que evitou uma duplicata em 40 dos 205 episódios); um servidor de tool não tinha como informar que o serviço por trás deu timeout; uma escrita em lote cortada ao meio não tinha como terminar (agora `batch`); e uma pessoa retomando uma pausa não tinha como dar a resposta da tool (agora `--uncertain done=<resposta>`). O lote agora verifica em vez de pausar, o que tem um custo: sob um commit atrasado ele duplica como toda outra escrita verificada.

6.9 Declarações erradas

O ponto fraco da abordagem é a própria declaração: se o programador declara um efeito errado, o compilador acredita. Declaramos cada uma das 22 tools do LIMBO de cada jeito possível (`read`, `write` sem chave, `write` com chave, `write once`) e classificamos cada declaração com os contratos internos do LIMBO. Uma declaração é perigosa se o runtime pode repetir uma escrita não idempotente.

Declaração perigosa	Avisada
Escrita não idempotente declarada `read`	10 de 10
Escrita não idempotente como `write` sem chave	10 de 10
`write` com chave num serviço que ignora chaves	**0 de 8**

Tabela 10: O que as anotações MCP pegam. 3 das 60 declarações seguras também receberam aviso (escritas idempotentes declaradas `read`).

As anotações pegam tratar uma escrita como leitura e esquecer a chave. Elas deixam passar o engano que mais pesou no LIMBO, confiar numa chave que o serviço ignora, porque as anotações MCP não têm vocabulário para chaves de idempotência [14]. Propomos uma ao MCP, `idempotencyKeyHint`, por tool (`docs/mcp/idempotency-key-hint.md`): com nosso adaptador do LIMBO declarando-a a partir de quais serviços respeitam a chave, a Calyx avisa (`W0703`) as 8 declarações desse tipo, sem novos falsos positivos. Quem declara é o adaptador, não o LIMBO: isso mostra o que a conferência faz quando os servidores dizem a verdade. Quando podem não dizer, `calyx check --tools --probe` testa a afirmação no ambiente de teste do serviço: duas chamadas com a mesma chave, depois uma contagem dos efeitos com uma tool de leitura. Contra o Stripe em modo de teste, aprova nosso servidor e acusa (`E0704`, dois reembolsos) uma versão dele que deixa de repassar a chave e continua anunciando `idempotencyKeyHint: true`.

6.10 Injeção de prompt e políticas (E6)

Um agente de atendimento com seis tools (ticket, pedido, cliente, reembolso, e-mail, endereço) atende 18 tickets: 6 benignos e 12 com uma instrução de um atacante no ticket ou nas observações do pedido (reembolsar 900, reembolsar outro pedido, mandar os dados do cliente para fora, trocar o endereço, consultar outro cliente, mandar um link de phishing). Os dois programas têm o mesmo prompt, com as regras escritas nele; um acrescenta

quatro políticas (Listagem 2). O servidor de tools registra toda chamada que recebe.

```
policy refund:
    require amount > 0
    require order from get_ticket.order
    deny in agent if amount > 100
policy send_email:
    require to from get_order.email
    require not ("://" in body)
```

Listagem 2: Duas das quatro políticas da E6.

Modelo, regras	Ataques	Benignos feitos	Recusadas
Obediente, no prompt	12 de 12	6 de 6	0
Obediente, `policy`	**0 de 12**	6 de 6	10
Gemini, no prompt	0 de 24	12 de 12	0
Gemini, `policy`	0 de 24	12 de 12	0

Tabela 11: Ataques que aconteceram. Obediente: um modelo falso que executa toda linha `CALL tool {...}` que lê, o pior caso. Gemini: `gemini-3.5-flash-lite`, 2 repetições. Nenhuma chamada legítima foi recusada.

Com o modelo obediente, as regras no prompt não seguram nada e as políticas seguram todos os ataques (Tabela 11); as duas trocas de endereço nem são tentadas, porque o agente com políticas não pode ter essa tool. O Gemini não caiu em nenhuma das 24 tentativas com as regras só no prompt e, com políticas, nunca tentou uma chamada que elas recusam: as políticas não custaram nada (mediana de 5,1 s contra 4,6 s por ticket, dentro da variação do provedor) e não fizeram nada. Nas duas versões, os mesmos dois tickets atacados (mandar os dados para fora; um link de phishing, os dois no texto do ticket) fizeram o Gemini largar também o reembolso legítimo: nenhum dano, mas nenhum atendimento, e políticas não evitam isso. Então a E6 não mostra que um modelo atual precisa de políticas contra esses ataques, que são uma frase cada; mostra que o que as políticas declaram vale, faça o modelo o que fizer.

6.11 Escrevendo um workflow real

Um exemplo de atendimento ao cliente (triagem tipada, um agente com tools só de leitura, reembolsos com aprovação e prazo, memória do cliente) rodou com o Gemini nos três caminhos e expôs dois problemas da linguagem, ambos corrigidos: a espera pela aprovação começava antes de a proposta existir, e não havia como mostrar à pessoa o que ela estava aprovando (agora `receive ... about`). Também expôs um limite que a linguagem não trata: o modelo prometeu ações que não fez.

7 Ameaças à validade

O mesmo autor dos dois lados. Os baselines, o corpus de bugs e os programas do LIMBO foram escritos pelo autor da Calyx. As tarefas, as falhas, o avaliador e os baselines publicados do LIMBO são de outros, e o adaptador do LIMBO usa só as tools públicas do LIMBO; a porta independente (E4) está pendente. **Provas.** T1–T3 são mecanizados em Lean só para uma chamada; para programas inteiros têm esboços de prova e uma verificação exaustiva limitada. Os dois modelos são escritos à mão, não extraídos do código. **Bugs achados na Calyx.** Vários bugs da Calyx foram achados medindo e corrigidos antes dos números relatados; dizemos onde. **Declarações erradas.** A conferência contra as anotações MCP só funciona para servidores que as enviam, e deixa passar chaves ignoradas. **Modelos.** A maioria dos experimentos usa modelos falsos; W1 e W2 foram repetidos com o Gemini com N pequeno, e na W2 o modelo sempre propôs o mesmo valor, então o risco de uma resposta diferente depois de uma queda não foi exercitado. Os ataques da E6 são frases fixas, não ajustadas ao modelo. **Ambiente.** Uma máquina; a W8 (várias máquinas) acrescenta latência com um proxy em vez de rodar contra um banco numa rede de verdade; só as rodadas do Stripe usam um serviço real pela rede, o resto usa serviços falsos; as versões são fixas. **Estudo de bugs reais.** Issues lidas pelos resumos, um classificador, uma amostra limitada pela busca do GitHub.

8 Trabalhos relacionados

Efeitos duplicados em agentes. O LIMBO [12] mede onde a garantia de efeito único deve morar e conclui que ela depende do contrato da tool. Outros estudos contam violações de idempotência sob novas tentativas [8] e avaliam a recuperação de resultados ambíguos de tools [19]; um invólucro de tools com verificação de pós-condição, chaves de idempotência e conferir antes de reenviar reduz duplicatas [13], o mesmo mecanismo do `verify` e das chaves da Calyx, oferecido como biblioteca opcional. A Calyx parte da mesma conclusão e torna o contrato obrigatório e verificado pelo compilador.

Transações e imposição em tempo de execução. O GoEX [16] defende desfazer e confinar danos; SagaLLM [4], Atomix [15], Cordon [5] e Agentic Transaction [20] oferecem execução transacional ou com compensação; o AgentRewind [25] faz checkpoint e rebobina agentes; o AgentSpec [22] impõe regras do usuário ao rodar. São mecanismos de runtime; a Calyx pede ao programa que declare o que eles precisam e confere isso antes de rodar. Suas compensações (`compensate`) são uma saga desse tipo, limitada aos perdedores de uma corrida e obrigada a ser escrita com chave. Suas políticas são regras como as do AgentSpec, que o compilador confere quanto a tipo, pureza e procedência e, quando os argumentos estão escritos no programa, já ao compilar (`E0723`).

Análise estática e cálculos para agentes. Trabalhos recentes analisam programas de agentes escritos em frameworks existentes quanto a propriedades estruturais [1] e não terminação [2], dão um cálculo tipado para composição de agentes [3], ou rastreiam o fluxo de informação em programas com LLM [7]. O CaMeL [6] se defende de injeção de prompt separando o fluxo de controle do fluxo de dados e marcando valores com capacidades; o `from` da Calyx é um caso mais restrito e declarado dessa ideia, conferido por argumento. A Calyx confere outra propriedade, o contrato dos efeitos externos, que os frameworks existentes não escrevem em lugar nenhum; seu núcleo formal poderia estender um cálculo desses com diários e quedas.

Frameworks e execução durável. O ReAct [24] intercala raciocínio e ação num laço guiado pelo modelo, que a Calyx oferece como construção limitada; o AutoGen [23] organiza agentes como conversas; o DSPy [10] otimiza prompts de pipelines tipados; o LangGraph [11] torna o grafo explícito e faz checkpoint de cada passo. O Temporal [21] e o Restate [17] registram cada passo e retomam sem refazê-lo; nos dois, a idempotência de um efeito externo é responsabilidade do programador. A Calyx empata com o Temporal na recuperação quando o cuidado é escrito, e difere por exigi-lo.

9 Conclusão

Medido contra o LangGraph e o Temporal em quedas, esperas e concorrência, o resultado é consistente: os baselines acertam quando o programador escreve o cuidado, e a Calyx exige que

ele seja escrito. Num benchmark externo, a Calyx empata com os melhores modelos quando as tools não aceitam chave e chega a zero duplicatas quando aceitam, o que apoia a tese de que a garantia mora no contrato e de que exigi-lo vale a pena. O custo do runtime é pequeno e seu paralelismo iguala o asyncio escrito à mão. A parte mais fraca da evidência é a que mais importa para a tese: se o compilador pega, antes de rodar, os bugs que programadores reais cometem. O corpus do autor diz que sim; as issues públicas não dizem nem sim nem não; uma porta independente é o próximo passo.

Disponibilidade. A Calyx, os experimentos e seus dados estão no repositório https://github.com/daltonfontes/calyx, cujo diretório `bench/` tem os scripts que produzem cada número deste paper.

Referências

[1] 2026. Agentproof. Recuperado de https://arxiv.org/abs/2603.20356

[2] 2026. When Agents Do Not Stop. Recuperado de https://arxiv.org/abs/2607.01641

[3] 2026. λ_A, a typed calculus for agent composition. Recuperado de https://arxiv.org/abs/2604.11767

[4] Edward Y. Chang e Longling Geng. 2025. SagaLLM: Context Management, Validation, and Transaction Guarantees for Multi-Agent LLM Planning. Recuperado de https://arxiv.org/abs/2503.11951

[5] Zheng Chen, Hanqing Liu, Duling Xu, Dong Dong, Jialin Li, Bangzheng Pu, e Jidong Zhai. 2026. Cordon: Semantic Transactions for Tool-Using LLM Agents. Recuperado de https://arxiv.org/abs/2606.17573

[6] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, e Florian Tramèr. 2025. Defeating Prompt Injections by Design. Recuperado de https://arxiv.org/abs/2503.18813

[7] Zac Garby, Andrew D. Gordon, e David Sands. 2026. The LLMbda Calculus. Recuperado de https://arxiv.org/abs/2602.20064

[8] Sanjana Gopnal Swamy. 2026. Do LLM Agents Act Exactly Once? Measuring Idempotency Violations Under Retries. Recuperado de https://github.com/gssanjana4/idempotencybench

[9] Pat Helland. 2012. Idempotence Is Not a Medical Condition. *ACM Queue* 10, 4 (2012), 30–46. https://doi.org/10.1145/2181796.2187821

[10] Omar Khattab e others. 2024. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. Em *International Conference on Learning Representations (ICLR)*, 2024.

[11] LangChain. 2026. LangGraph. Recuperado de https://github.com/langchain-ai/langgraph

[12] Jiapeng Li. 2026. LIMBO: Benchmark code, harness adapters and episode traces. Recuperado de https://github.com/jaxblack/limbo-bench

[13] Isham Kalappurackal Mansoor, Abhishek Phadke, e Pratip Rana. 2026. Verified Tool Calls Improve LLM Agent Reliability Under Non-Atomic Failures. Recuperado de https://arxiv.org/abs/2608.02645

[14] Model Context Protocol. 2025. Model Context Protocol Specification, Version 2025-06-18: Schema Reference (ToolAnnotations). Recuperado de https://modelcontextprotocol.io/specification/2025-06-18/schema

[15] Bardia Mohammadi, Nearchos Potamitis, Lars Klein, Akhil Arora, e Laurent Bindschaedler. 2026. Atomix: Timely, Transactional Tool Use for Reliable Agentic Workflows. Recuperado de https://arxiv.org/abs/2602.14849

[16] Shishir G. Patil, Tianjun Zhang, Vivian Fang, Noppapon C., Roy Huang, Aaron Hao, Martin Casado, Joseph E. Gonzalez, Raluca Ada Popa, e Ion Stoica. 2024. GoEX: Perspectives and Designs Towards a Runtime for Autonomous LLM Applications. Recuperado de https://arxiv.org/abs/2404.06921

[17] Restate. 2026. Restate. Recuperado de https://restate.dev/

[18] Stripe. 2026. Idempotent requests. Recuperado de https://docs.stripe.com/api/idempotent_requests

[19] Shengyao Sun. 2026. Did It Happen? Counterfactual Evaluation of LLM Agent Recovery from Ambiguous Tool Outcomes. https://doi.org/10.21203/rs.3.rs-10730245/v1

[20] Zhaoyan Sun, Xiaoxiao Wang, e Guoliang Li. 2026. Agentic Transaction: Towards ACID-Compliant Agent Systems. Recuperado de https://arxiv.org/abs/2608.13900

[21] Temporal Technologies. 2026. Temporal. Recuperado de https://temporal.io/

[22] Haoyu Wang, Christopher M. Poskitt, e Jun Sun. 2026. AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents. Em *Proc. 48th IEEE/ACM International Conference on Software Engineering (ICSE)*, 2026. 2938–2950.

[23] Qingyun Wu e others. 2024. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. Em *Conference on Language Modeling (COLM)*, 2024.

[24] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, e Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. Em *International Conference on Learning Representations (ICLR)*, 2023.

[25] Yu Zhuang, Kefei Chen, Yitong Duan, Shuxin Zheng, Jian Li, e Xu-Yao Zhang. 2026. AgentRewind: Recoverable Execution for Long-Horizon LLM Agents. Recuperado de https://arxiv.org/abs/2608.14380